

第 5 次作业

算法设计与分析

姓名	倪伟丰	学号	2024110089
班级	24 大数据 2 班	教师	韩恺
提交日期	2026 年 4 月 4 日		

1 Question 1

You have maps of parts of the space station, each starting at a prison exit and ending at the door to an escape pod. The map is represented as a matrix of 0s and 1s, where 0s are passable space and 1s are impassable walls. The door out of the prison is at the top left $(0, 0)$ and the door into an escape pod is at the bottom right $(w - 1, h - 1)$. Write an algorithm that generates the length of a shortest path from the prison door to the escape pod, where you are allowed to remove one wall as part of your remodeling plans.

解答：

以下统一把状态写成“行、列”坐标 (x, y) ，其中 $0 \leq x < h$ ， $0 \leq y < w$ ，因此终点记为 $(h - 1, w - 1)$ 。为处理“最多移除一堵墙”的限制，可以在普通 BFS 的基础上加入一维状态，记录是否已经使用过拆墙机会。

- 维护数组 $D[0 \dots h - 1][0 \dots w - 1][0, 1]$ ，记录从起点到每个状态的最短距离，初始值均为 $+\infty$ 。
- 定义状态 (x, y, k) ：当前位置为 (x, y) ，其中 $k = 0$ 表示尚未拆墙， $k = 1$ 表示已经拆过一堵墙。
- 维护队列 Q 做 BFS。初始时把 $(0, 0, 0)$ 入队，并令 $D[0][0][0] = 0$ 。
- 每次从队首取出状态 (x, y, k) 。若 $(x, y) = (h - 1, w - 1)$ ，则直接返回 $D[x][y][k]$ 。
- 枚举四个相邻位置 (nx, ny) 。若它在网格内、且 $G[nx][ny] = 0$ ，并且状态 (nx, ny, k) 尚未访问，则将其入队，并令

$$D[nx][ny][k] = D[x][y][k] + 1.$$

- 若 (nx, ny) 是墙，即 $G[nx][ny] = 1$ ，并且当前 $k = 0$ ，那么可以把这堵墙作为唯一一次拆墙操作，转移到状态 $(nx, ny, 1)$ ；若该状态尚未访问，则入队并令

$$D[nx][ny][1] = D[x][y][0] + 1.$$

7. 若队列最终为空仍未到达终点，则返回 -1 。

时间复杂度：状态总数为 $2wh$ ，因为每个格子对应“未拆墙/已拆墙”两种状态。每个状态至多出队一次，每次只检查 4 个方向，因此总时间复杂度为 $O(wh)$ ，空间复杂度也为 $O(wh)$ 。

Algorithm 1 允许破墙一次的 BFS 算法

Require: 网格地图 G ，大小为 $h \times w$ ，其中 $G[i][j] \in \{0, 1\}$

```

1: 初始化队列  $Q \leftarrow \emptyset$ 
2: 初始化访问数组  $V[0 \dots h-1][0 \dots w-1][0, 1] \leftarrow \text{false}$ 
3: 初始化距离数组  $D[0 \dots h-1][0 \dots w-1][0, 1] \leftarrow \infty$ 
4:  $Q.\text{enqueue}((0, 0, 0))$ 
5:  $V[0][0][0] \leftarrow \text{true}$ 
6:  $D[0][0][0] \leftarrow 0$ 
7: 定义方向数组  $\Delta \leftarrow \{(0, 1), (0, -1), (1, 0), (-1, 0)\}$ 
8: while  $Q \neq \emptyset$  do
9:    $(x, y, k) \leftarrow Q.\text{dequeue}()$ 
10:  if  $(x, y) = (h-1, w-1)$  then
11:    return  $D[x][y][k]$ 
12:  end if
13:  for 每个方向  $(dx, dy) \in \Delta$  do
14:     $nx \leftarrow x + dx, \quad ny \leftarrow y + dy$ 
15:    if  $0 \leq nx < h$  and  $0 \leq ny < w$  then
16:      if  $G[nx][ny] = 0$  and  $\neg V[nx][ny][k]$  then
17:         $V[nx][ny][k] \leftarrow \text{true}$ 
18:         $D[nx][ny][k] \leftarrow D[x][y][k] + 1$ 
19:         $Q.\text{enqueue}((nx, ny, k))$ 
20:      else if  $G[nx][ny] = 1$  and  $k = 0$  and  $\neg V[nx][ny][1]$  then
21:         $V[nx][ny][1] \leftarrow \text{true}$ 
22:         $D[nx][ny][1] \leftarrow D[x][y][0] + 1$ 
23:         $Q.\text{enqueue}((nx, ny, 1))$ 
24:      end if
25:    end if
26:  end for
27: end while
28: return  $-1$ 

```

正确性证明概要

先构造一个状态图 H ：图中的每个顶点都是一个状态 (x, y, k) ，其中 $k \in \{0, 1\}$ 表示是否已经使用过拆墙机会。若从状态 (x, y, k) 可以一步移动到相邻格 (nx, ny) ，则在

H 中连一条单位长度的边：若 $G[nx][ny] = 0$ ，则连到 (nx, ny, k) ；若 $G[nx][ny] = 1$ 且 $k = 0$ ，则连到 $(nx, ny, 1)$ 。

引理 1 (状态图与原问题等价). 原迷宫中任意一条从起点到终点、且至多拆除一堵墙的合法路径，都与状态图 H 中一条从 $(0, 0, 0)$ 到某个终点状态 $(h-1, w-1, k)$ 的路径一一对应，且两者长度相同。

证明. 原问题中每走一步，在状态图中都对应一次状态转移；走到空地时 k 不变，第一次穿过墙时 k 从 0 变为 1。反过来，状态图中的每条合法路径也都精确记录了在原网格中的移动过程以及是否使用过拆墙机会。由于每次状态转移都对应原问题中的一步移动，所以两者路径长度完全相同。 $\square$

引理 2 (BFS 最优性). 对于状态图 H 中的任意状态 (x, y, k) ，BFS 首次将其出队时，记录的距离 $D[x][y][k]$ 就是从 $(0, 0, 0)$ 到该状态的最短路径长度。

证明. 状态图 H 的每条边长度都为 1，因此 BFS 会按距离从小到大逐层扩展。又因为数组 V 保证每个状态至多入队一次，所以某状态第一次出队时的距离就是到该状态的最短距离，这是 BFS 在单位权图上的标准性质。 $\square$

由上面两个引理可知，原问题的答案等于状态图中从 $(0, 0, 0)$ 到两个终点状态 $(h-1, w-1, 0)$ 、 $(h-1, w-1, 1)$ 的最短距离中的较小者。算法在首次出队某个终点状态时立刻返回 $D[x][y][k]$ ；这是正确的，因为 BFS 的出队顺序按距离非减排列，若另一个终点状态有更短距离，它必然会更早出队。若队列为空仍未出队任何终点状态，则两个终点状态都不可达，故原问题无解，返回 -1 正确。

2 Question 2

Consider a directed graph $G = (V, E)$ with nonnegative edge lengths and a starting vertex s . Define the *bottleneck* of a path to be the maximum length of one of its edges (as opposed to the sum of the lengths of its edges).

Show how to modify Dijkstra's algorithm to compute, for each vertex $v \in V$, the smallest bottleneck of any s - v path. Your algorithm should run in $O(m \cdot n)$ time, where m and n denote the number of edges and vertices, respectively. You must also prove the correctness of your algorithm.

解答：

仍然保留 Dijkstra 的贪心思想：维护一个“已确定最优瓶颈值”的顶点集合 S ，每一轮都把当前瓶颈估计最小的未确定顶点加入 S 。为了严格满足题目要求的 $O(mn)$ 时间，这里每轮直接扫描全部边来更新跨越割 $(S, V \setminus S)$ 的候选值，而不再额外扫描所有顶点寻找最小值。

1. 维护数组 $B[1 \dots n]$, 其中 $B[v]$ 表示当前已知的从 s 到 v 的最小瓶颈估计值; 初始时 $B[s] = 0$, 其余顶点均为 $+\infty$ 。
2. 维护布尔数组 $V[1 \dots n]$, 其中 $V[v] = \mathbf{true}$ 表示顶点 v 已经被加入集合 S , 其瓶颈值已经确定。初始时仅令 $V[s] = \mathbf{true}$ 。
3. 重复至多 $n - 1$ 轮。在每一轮中, 扫描所有边 $(x, y) \in E$:
 - 若 $x \in S$ 且 $y \notin S$, 则经过这条边到达 y 的候选瓶颈值为
$$\max(B[x], w(x, y)).$$
 - 用该值更新 $B[y]$ 。
 - 在本轮扫描过程中, 同时记录所有未确定顶点中当前 B 值最小的那个顶点 u 。
4. 若本轮结束后不存在这样的顶点 u , 则说明其余顶点都不可达, 算法终止。
5. 否则将 u 加入 S , 即令 $V[u] = \mathbf{true}$, 继续下一轮。
6. 算法结束后返回数组 B ; 其中 $B[v]$ 即为从 s 到 v 的最小瓶颈值, 不可达时为 $+\infty$ 。

Algorithm 2 最小瓶颈路径算法**Require:** 有向图 $G = (V, E)$, 非负边权函数 w , 起点 $s \in V$, 其中 $n = |V|, m = |E|$ **Ensure:** 数组 $B[1 \dots n]$, $B[v]$ 表示从 s 到 v 的最小瓶颈值

```

1: 初始化  $B[1 \dots n] \leftarrow +\infty$ 
2: 初始化  $V[1 \dots n] \leftarrow \text{false}$ 
3:  $B[s] \leftarrow 0$ 
4:  $V[s] \leftarrow \text{true}$ 
5: for  $i = 1$  to  $n - 1$  do
6:    $u \leftarrow -1, best \leftarrow +\infty$ 
7:   for 每条边  $(x, y) \in E$  do
8:     if  $V[x] = \text{true}$  and  $V[y] = \text{false}$  then
9:        $cand \leftarrow \max(B[x], w(x, y))$ 
10:      if  $cand < B[y]$  then
11:         $B[y] \leftarrow cand$ 
12:      end if
13:      if  $B[y] < best$  then
14:         $best \leftarrow B[y], u \leftarrow y$ 
15:      end if
16:    end if
17:  end for
18:  if  $u = -1$  then
19:    break
20:  end if
21:   $V[u] \leftarrow \text{true}$ 
22: end for
23: return  $B$ 

```

算法适用性说明:

虽然目标函数从“路径边权和最小”变成了“路径最大边权最小”，但贪心定点思想仍然成立：一旦某个未确定顶点在当前所有候选中具有最小的瓶颈估计值，它的值就已经是全局最优的，不会被后续路径继续改进。因此仍可沿用 Dijkstra 的“逐步扩大已确定集合”的框架，只需把松弛公式改为

$$B[y] \leftarrow \min(B[y], \max(B[x], w(x, y))).$$

时间复杂度分析:

主循环至多执行 $n - 1$ 轮；每一轮只扫描全部 m 条边一次，因此主过程耗时为 $O(mn)$ 。初始化需要 $O(n)$ 时间；当 $m > 0$ 时， $O(n)$ 被 $O(mn)$ 吸收，因此总时间复杂度记为 $O(mn)$ 。若 $m = 0$ ，则算法显然在线性时间 $O(n)$ 内结束。

正确性证明:

记 $\beta(v)$ 为从 s 到 v 的最小瓶颈值。设

$$S = \{v \in V \mid V[v] = \text{true}\}.$$

引理 3 (循环不变量). 在每一轮扫描所有边结束后, 始终有:

1. 对任意 $u \in S$, 都有 $B[u] = \beta(u)$;
2. 对任意 $v \notin S$, $B[v]$ 等于所有满足“除终点 v 外, 其余顶点都在 S 中, 且最后一条边从 S 进入 v ”的 $s \rightarrow v$ 路径的最小瓶颈值; 若不存在这类路径, 则 $B[v] = +\infty$ 。

证明. 初始化时, $S = \{s\}$, 显然 $B[s] = 0 = \beta(s)$, 故第 1 条成立。对任意 $v \neq s$, 此时只允许使用形如 $s \rightarrow v$ 的单边路径, 因此扫描所有边后得到的 $B[v]$ 恰好是所有“最后一条边从 S 进入 v ”的路径中的最小瓶颈值, 第 2 条也成立。

设某轮开始前不变量成立, 且本轮选出的顶点为 u 。把 u 加入 S 后, 任何通往未确定顶点 v 的新允许路径, 要么原先就只经过旧的 S , 要么最后一步经过新加入的顶点 u 进入 v 。算法本轮重新扫描全部边, 并用

$$\max(B[x], w(x, y))$$

更新所有从 S 指向 $V \setminus S$ 的边, 因此新的 $B[v]$ 仍恰好等于上述那一类路径的最小瓶颈值。不变量得证。□

引理 4 (贪心选择性质). 若某一轮选出的顶点 $u \notin S$ 满足 $B[u] = \min_{v \notin S} B[v]$, 则有 $B[u] = \beta(u)$ 。

证明. 由循环不变量第 2 条, 若 $B[u] < +\infty$, 则必定存在一条从 s 到 u 的实际路径, 其瓶颈值恰为 $B[u]$, 因此

$$\beta(u) \leq B[u].$$

另一方面, 任取任意一条从 s 到 u 的路径 P 。设 x 是 P 上第一个不在 S 中的顶点, y 是它在 P 上的前驱, 则 $y \in S$ 。由于 P 的瓶颈值至少为经过边 (y, x) 时的瓶颈值, 有

$$\text{bottleneck}(P) \geq \max(\beta(y), w(y, x)).$$

由循环不变量第 1 条, $\beta(y) = B[y]$; 再由第 2 条可知

$$B[x] \leq \max(B[y], w(y, x)) = \max(\beta(y), w(y, x)).$$

由于算法在本轮选择的是未确定顶点中 B 值最小者, 所以 $B[u] \leq B[x]$ 。于是

$$B[u] \leq B[x] \leq \max(\beta(y), w(y, x)) \leq \text{bottleneck}(P).$$

这对任意路径 P 都成立, 因此 $B[u] \leq \beta(u)$ 。结合上式 $\beta(u) \leq B[u]$, 得到 $B[u] = \beta(u)$ 。□

算法正确性：

对加入集合 S 的轮数作归纳。初始时只有 s 在 S 中，且 $B[s] = 0 = \beta(s)$ 。若归纳假设当前 S 中所有顶点都已满足 $B[v] = \beta(v)$ ，则下一轮选出的顶点 u 由上一个引理可知也满足 $B[u] = \beta(u)$ 。因此，每次加入 S 的顶点都会被正确地定值。算法终止时，所有可达顶点都已经被加入过 S 并正确赋值；未被加入的顶点始终保持 $+\infty$ ，恰好表示不可达。故最终返回的数组 B 正是题目要求的最小瓶颈值数组，即

$$B[v] = \min_{P: s \rightsquigarrow v} \max_{e \in P} w(e).$$